75-FOC 28

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。
我们开始上课啊，上次课，我们分析了切无感闭环的条件。

```
if(tFoc.Direction == CCW)
    tCore.qStartThetaDelta = tObs.qTheta - tIm.qParkAngle;//计算逆时针转动估
else
    tCore.qStartThetaDelta = tIm.qParkAngle - tObs.qTheta;
```

也就是说，我们去判断这个角度差范围，如果连续多次满足在 60°~120°这个角度范围内，则就可以切。我们的课程因为是空载，所以启动时只要加上 Iq,电机的瞬时速度很快就会上来，如果是重载的话，这个条件可以再加上一个速度阀值判断，超过某个转速再来判断这个条件。上面的角度差呢，已经求解出来了，下面需要的就是判断这个角度差的范围。

```
    tCore.qEAngle = tIm.qParkAngle - EANGLE90 - tUm.qDQAngle;

UsOut();

if (tIm.qParkAngle > EANGLE60)
    tCore.StartSwitchNum = 1;

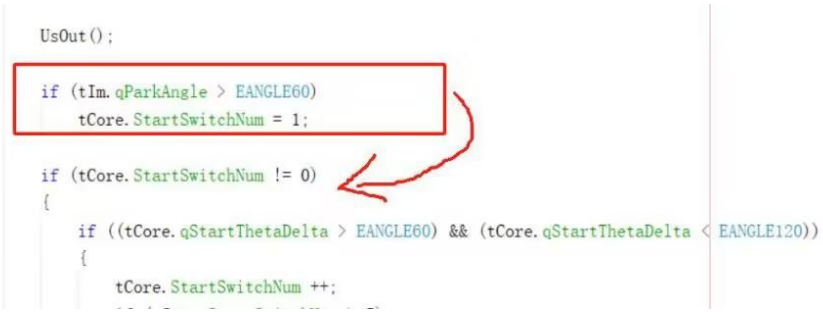
if (tCore.StartSwitchNum != 0)
{
    if ((tCore.qStartThetaDelta > EANGLE60) && (tCore.qStartThetaDelta < EANGLE120))
    {
        tCore.StartSwitchNum ++;
        if (tCore.StartSwitchNum > 5)
            tFoc.Status = WORK_STATE;
    }
    else
    {
        tCore.StartSwitchNum = 1;
    }
}
}
```

大家可以看下，这里是判断这个角度差连续满足在 60°~120°这个范围超过 5 次。如果条件成立的话，就会切为无感。当然了，判断这个角度差的话，肯定不是一到这个强拖运行状态再判断。而是要等电机开始转动到一定的角度之后，再来判断，也就是说这时候瞬时转速会达到使观测器收敛。

问：角度差等于 0 不是更好 为什么不是大于 0?

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

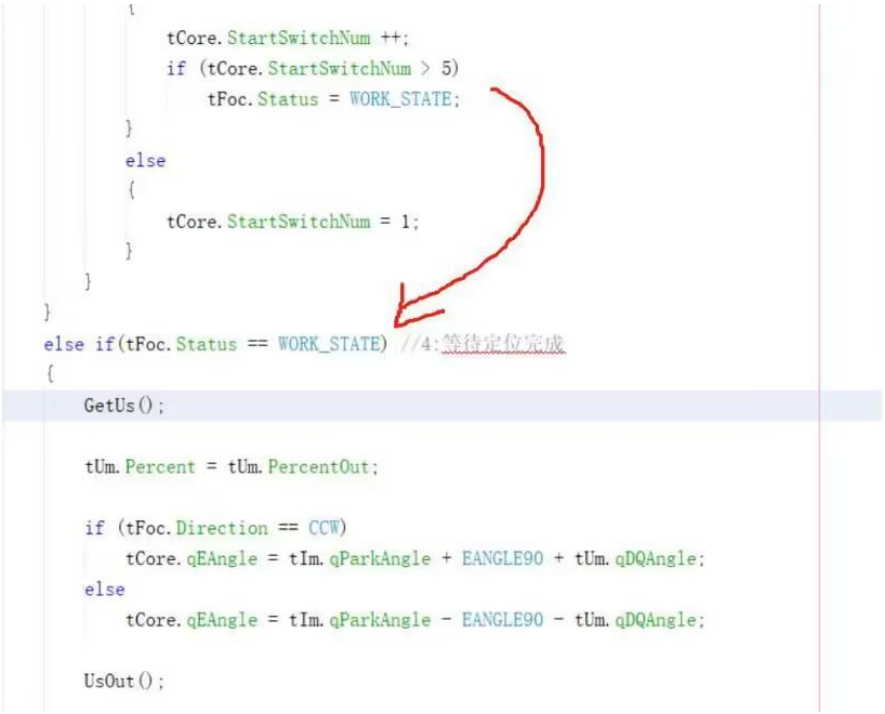
答：角度差要在 90 度附近啊，因为这时候定子磁场和转子磁场夹角才是 90 度，力矩最大。



这里的满足必须是连续的 5 次，倘若有 1 次不满足，都会重新去判断。当然了究竟是 5 次还是 4 次，3 次可根据调试确定。所以，切换的条件，还是相对严格的，可能还需要根据实际情况去调整。这样的话，才能保证切换到无感状态后，电机也能稳定运行。 换句话说，在正常加速条件下，有可能有某段时间窗口正好满足切换条件，就切换进去了。

问：这个角度差应该是跳跃的吧，次数也不能太多是么？
答：这个角度差是随着加速的过程变化的，次数太多有可能不能满足切闭环条件。

问：两个值都在 0-360 之间变化，所以这个差值我理解应该是过 0 度的时候会跳变一次吧？？
答：这个差值正常情况下会从 0 度逐渐增大到 90 度，当然低速时角度是不准确的，所以要尽量高于某个转速开始检测。



本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

切到无感运行状态之后，一样的需要获取 SVPWM 模块的输入角度跟相应的电压矢量的模长。只不过呢，切入进来之后，转子的位置角就不再由我们人为给定了，而是靠估算的反电动势来计算角度。

```
else if(tFoc.Status == WORK_STATE) //4:等待定位完成
{
    GetUs();

    tUm.Percent = tUm.PercentOut;

    if (tFoc.Direction == CCW)
        tCore.qEAngle = tIm.qParkAngle + EANGLE90 + tUm.qDQAngle;
    else
        tCore.qEAngle = tIm.qParkAngle - EANGLE90 - tUm.qDQAngle;

    UsOut();
```

这段代码的作用呢，就是切入进来之后，发出 SVPWM 调制波，继续驱动电机运行。

```
    GetUs();

    tUm.Percent = tUm.PercentOut;

    if (tFoc.Direction == CCW)
        tCore.qEAngle = tIm.qParkAngle + EANGLE90 + tUm.qDQAngle;
    else
        tCore.qEAngle = tIm.qParkAngle - EANGLE90 - tUm.qDQAngle;

    UsOut();

    if(tFoc.Direction == CCW)
        tIm.qParkAngle = tObs.qTheta - tCore.qStartThetaDelta;
    else
        tIm.qParkAngle = tCore.qStartThetaDelta + tObs.qTheta;
```

然后，红框中这段代码呢，是用估算的反电动势合成矢量角，来计算估算的转子位置角。tObs.qTheta 是观测器估算出来的反电动势合成矢量角。

刚切进来的时候，tCore.qStartThetaDelta 是估算角度跟人为给定的转子位置角的差值。这个差值呢，我们也说了，调节到它是 60°~120°范围内，而我们期望这个差值呢是 90°。所以呢，为了平滑过度到 90°，进入闭环后我们可以让这个角度差慢慢变到 90°。

```
    if(tFoc.Direction == CCW)
        tIm.qParkAngle = tObs.qTheta - tCore.qStartThetaDelta;
    else
        tIm.qParkAngle = tCore.qStartThetaDelta + tObs.qTheta;

    if (tCore.qStartThetaDelta < EANGLE90)
    {
        tCore.qStartThetaDelta ++;
    }
    else if (tCore.qStartThetaDelta > EANGLE90)
    {
        tCore.qStartThetaDelta --;
    }
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

调整的思路呢，就是这种加加，减减的方式。最终呢，目标是把角度差调整到 90° 。当然了，这里肯定不是仅仅是数值上的加加减减，而是会实际作用到电机的。

```
if (tFoc.Direction == CCW)
    tCore.qEAngle = tIm.qParkAngle + EANGLE90 + tUm.qDQAngle;
else
    tCore.qEAngle = tIm.qParkAngle - EANGLE90 - tUm.qDQAngle;

UsOut();

if(tFoc.Direction == CCW)
    tIm.qParkAngle = tObs.qTheta - tCore.qStartThetaDelta;
else
    tIm.qParkAngle = tCore.qStartThetaDelta + tObs.qTheta;

if (tCore.qStartThetaDelta < EANGLE90)
{
    tCore.qStartThetaDelta ++;
}
else if (tCore.qStartThetaDelta > EANGLE90)
{
    tCore.qStartThetaDelta --;
}
```

大家看下，数值上的角度差的加加或减减，会让 ParkAngle 变化。这个 ParkAngle 变化，会调整电压合成矢量角。

通过上面代码这个调整的话，慢慢的可以让定子磁场跟转子磁场之间的夹角过渡到 90° 。

问：这里的角度变量傻傻的有点分不清哪个是哪个？

答：

电压合成矢量角度 = 转子N方向角度(d轴角度) + 90度 + 电压合成矢量与q轴夹角

对应中文表述可对照看上面公式。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
else if(tFoc.Status == WORK_STATE) //4:等待定位完成
{
    GetUs();

    tUm.Percent = tUm.PercentOut;

    if (tFoc.Direction == CCW)
        tCore.qEAngle = tIm.qParkAngle + EANGLE90 + tUm.qDQAngle;
    else
        tCore.qEAngle = tIm.qParkAngle - EANGLE90 - tUm.qDQAngle;

    UsOut();

    if(tFoc.Direction == CCW)
        tIm.qParkAngle = tObs.qTheta - tCore.qStartThetaDelta;
    else
        tIm.qParkAngle = tCore.qStartThetaDelta + tObs.qTheta;

    if (tCore.qStartThetaDelta < EANGLE90)
    {
        tCore.qStartThetaDelta ++;
    }
    else if (tCore.qStartThetaDelta > EANGLE90)
    {
        tCore.qStartThetaDelta --;
    }
```

上面这段代码的目的，就是切换到无感运行之后，要调整定子磁场跟转子磁场的夹角。调整到 90° 后，就可以让定子磁场跟转子磁场夹角是 90° 。这样的话，电机也就按照我们期望的方式，Id=0 控制，两个磁场夹角 90° ，电机转动的力矩是最大的。

实质上，到目前为止呢，电机都还是在仅有电流环控制模式下运行。如果我们要想控制电机的转速的话，就需要加上速度环。所以啊，下面我们就来分析下，速度环相关的代码。

```
    if (tCore.qStartThetaDelta < EANGLE90)
    {
        tCore.qStartThetaDelta ++;
    }
    else if (tCore.qStartThetaDelta > EANGLE90)
    {
        tCore.qStartThetaDelta --;
    }
    else
    {
        tCore.SpeedLoopStart = 1;
        if (tIm.qIqGiven < tCore.qSetIq)
            tIm.qIqGiven ++;
        if (tIm.qIqGiven > tCore.qSetIq)
            tIm.qIqGiven --;
    }
```

大家看下，当角度差被调整为 90° 之后，就会进入到 else 分支。进入到 else 分支后，也就说明定子磁场跟转子磁场的夹角是 90° 了。所以，如果我们要想进行速度控制，则就可以在进入到这个 else 分支后，把速度环开始的标志位置 1。

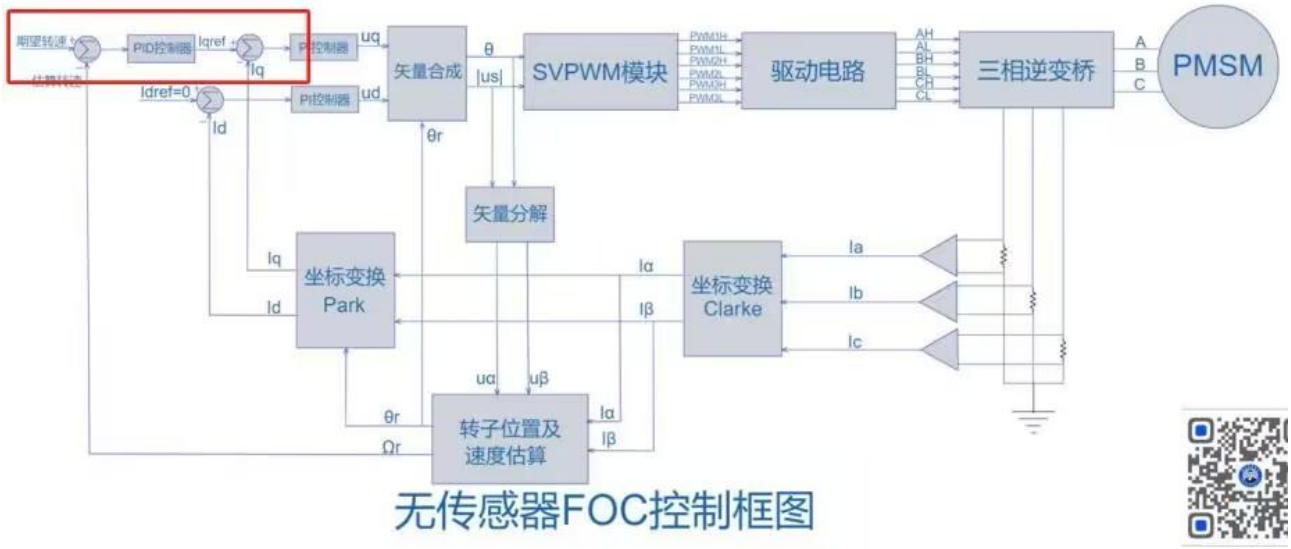
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
        else
        {
            tCore.SpeedLoopStart = 1;
            if (tIm.qIqGiven < tCore.qSetIq)
                tIm.qIqGiven ++;
            if (tIm.qIqGiven > tCore.qSetIq)
                tIm.qIqGiven --;
        }
    }
}
```

而这段代码，是给定一个速度环输出值与电流环 Iq 给定值的设定关系。

问：这里直接把设定值赋予给定值 可不可以？ 为什么要慢慢给？

答：为了速度变化更丝滑一点，而且因为这个控制周期比较短，是 50us，速度环的控制周期比较长，一般是 ms 级别。



无传感器FOC控制框图

大家来看下这幅图，我们如果要进行速度控制的话，需要设定一个期望转速，并且呢还要知道电机的实际转速。而这个期望转速呢，可以通过电位器来进行设定。估算转速的话，是来自于电流观测器。也就是，实际转速是靠观测器估算出来的。

所以啊，进行速度控制需要的两个速度量我们都获取到了。把两个速度值求一个差，然后输入到 PID 控制器，这样的话，就可以调整转速了。当然了，我们在实际的代码实现的时候，需要把估算转速、期望转速给获取到，最后再调用 PID 控制函数，来实现速度控制。

之前呢，电流环控制的时候我们是用汇编实现的 PI 控制，这里呢就写一个 C 语言的 PID 函数。这里呢，我们来分析下 C 语言写的 PID 控制。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
typedef struct
{
    signed int sErr;
    signed int sErrLast;
    signed int sLimit;
    UNION_S32 qSumErr32;
    unsigned int Kp;
    unsigned int Ki;
    unsigned int Kd;
    signed int sOut;
}tPID_Type;
```

这里呢，我们先定义一个结构体。然后呢，根据 PID 的公式，可以实现对应的函数。

问：这个 pid 函数能不能把积分部分去掉就 pi 控制了？

答：可以的，我们前面讲过，如果控制量是恒定的或者变化比较小，可以用 PI。如果控制量是变化的，则需要加上 D。

```
static signed int CalProportion(tPID_Type *tVar)
{
    UNION_S32 sTemp32;
    signed int sTemp;

    tPID_Type tPVar;
    tPVar = *tVar;

    sTemp32.S32 = __builtin_mulus(tPVar.Kp,tPVar.sErr);
    sTemp = sTemp32.Words.High;
    if (sTemp >= 0)
        sTemp32.Words.High = 32767 - sTemp;
    else
        sTemp32.Words.High = 32767 + sTemp;
    sTemp32.Words.Low = 0;
    tPVar.sLimit = sTemp32.S32;

    *tVar = tPVar;

    return (sTemp);
}
```

大家来看下，这是计算 PID 中的比例项的输出值，也就是计算 Kp*Err。

问：为什么要 32767-stemp?

答：为了计算限制值，P 和 I 有个总的限制值 32767，相当于 1，32767-stemp 就是计算留给积分项的限制值。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*==========================================================
static signed int CalProportion(tPID_Type *tVar)
{
    UNION_S32 sTemp32;
    signed int sTemp;

    tPID_Type tPVar;
    tPVar = *tVar;

    sTemp32.S32 = __builtin_mulus(tPVar.Kp,tPVar.sErr);
    sTemp = sTemp32.Words.High;
```

这里呢，是定义了一个结构体临时变量。然后呢，我们下面的计算，都是操作临时结构体，计算完后，再赋值回原来的结构体。

```
static signed int CalProportion(tPID_Type *tVar)
{
    UNION_S32 sTemp32;
    signed int sTemp;

    tPID_Type tPVar;
    tPVar = *tVar;

    sTemp32.S32 = __builtin_mulus(tPVar.Kp,tPVar.sErr);
    sTemp = sTemp32.Words.High;
    if (sTemp >= 0)
        sTemp32.Words.High = 32767 - sTemp;
    else
        sTemp32.Words.High = 32767 + sTemp;
```

大家看下，这里就是计算 Kp*Err，这里呢，相当于是计算 Q15*Q16，然后要得到 Q15 结果的话，就取高 16 位。

```
    tPID_Type tPVar;
    tPVar = *tVar;

    sTemp32.S32 = __builtin_mulus(tPVar.Kp,tPVar.sErr);
    sTemp = sTemp32.Words.High;
    if (sTemp >= 0)
        sTemp32.Words.High = 32767 - sTemp;
    else
        sTemp32.Words.High = 32767 + sTemp;
    sTemp32.Words.Low = 0;
    tPVar.sLimit = sTemp32.S32;

    *tVar = tPVar;

    return (sTemp);
}
```

然后，这段代码呢，是计算 PID 控制器的输出限幅值减去比例项的值。计算出来这个值的话，是作为后面积分项的限幅值使用的。因为这个部分呢，都是 Q 格式的运算，PID 控制器的输出呢，最大是 1，比例项占了一部分，剩下的就是积分项或者微分项的了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

问：限制值一般设置多少？有什么考究吗？
答：我们这里就是 1，因为我们是采用归一化的思路。

```
static signed int CalProportion(tPID_Type *tVar)
{
    UNION_S32 sTemp32;
    signed int sTemp;

    tPID_Type tPVar;
    tPVar = *tVar;

    sTemp32.S32 = __builtin_mulus(tPVar.Kp,tPVar.sErr);
    sTemp = sTemp32.Words.High;
    if (sTemp >= 0)
        sTemp32.Words.High = 32767 - sTemp;
    else
        sTemp32.Words.High = 32767 + sTemp;
    sTemp32.Words.Low = 0;
    tPVar.sLimit = sTemp32.S32;

    *tVar = tPVar;

    return (sTemp);
}
```

```
static signed int CalIntegration(tPID_Type *tVar)
{
    UNION_S32 sTemp32;

    tPID_Type tIVar;
    tIVar = *tVar;

    sTemp32.S32 = __builtin_mulus(tIVar.Ki,tIVar.sErr);
    tIVar.qSumErr32.S32 += sTemp32.S32;

    if (tIVar.qSumErr32.S32 > tIVar.sLimit)
        tIVar.qSumErr32.S32 = tIVar.sLimit;

    if (tIVar.qSumErr32.S32 < -tIVar.sLimit)
        tIVar.qSumErr32.S32 = -tIVar.sLimit;

    sTemp32.S32 = tIVar.qSumErr32.S32;

    *tVar = tIVar;
    return (sTemp32.Words.High);
}
```

然后，接着是积分，积分项的话，就是先计算 Ki*sErr，然后，再累加起来。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*==========================================================
static signed int CalIntegration(tPID_Type *tVar)
{
    UNION_S32 sTemp32;

    tPID_Type tIVar;
    tIVar = *tVar;

    sTemp32.S32 =  __builtin_mulus(tIVar.Ki,tIVar.sErr);
    tIVar.qSumErr32.S32 += sTemp32.S32;

    if (tIVar.qSumErr32.S32 > tIVar.sLimit)
        tIVar.qSumErr32.S32 = tIVar.sLimit;

    if (tIVar.qSumErr32.S32 < -tIVar.sLimit)
        tIVar.qSumErr32.S32 = -tIVar.sLimit;

    sTemp32.S32 = tIVar.qSumErr32.S32;

    *tVar = tIVar;
    return (sTemp32.Words.High);
}
```

然后呢，再判断累加值是否超过限幅值。如果大于正的限幅值，就等于正的限幅值，如果小于负的限幅值，就等于负的限幅值。

```
    tPID_Type tIVar;
    tIVar = *tVar;

    sTemp32.S32 = __builtin_mulus(tIVar.Ki,tIVar.sErr);
    tIVar.qSumErr32.S32 += sTemp32.S32;

    if (tIVar.qSumErr32.S32 > tIVar.sLimit)
        tIVar.qSumErr32.S32 = tIVar.sLimit;

    if (tIVar.qSumErr32.S32 < -tIVar.sLimit)
        tIVar.qSumErr32.S32 = -tIVar.sLimit;

    sTemp32.S32 = tIVar.qSumErr32.S32;

    *tVar = tIVar;
    return (sTemp32.Words.High);
}
```

最后呢，就更新积分累加项，这样的话，积分累加项就计算好了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*==========================================================================*/
static signed int CalDifferential(tPID_Type *tVar)
{
    UNION_S32 sTemp32;
    signed int sDeltaErr;

    tPID_Type tDVar;
    tDVar = *tVar;

    sDeltaErr = tDVar.sErr - tDVar.sErrLast;
    tDVar.sErrLast = tDVar.sErr;

    sTemp32.S32 = __builtin_mulus(tDVar.Kd, sDeltaErr);
    *tVar = tDVar;
    return (sTemp32.Words.High);
}
/*==========版权归张飞实战电子所有，违者必究！！！==========*/
```

好，接着我们来看下计算微分项的函数。

```
/*==========================================================*/
static signed int CalDifferential(tPID_Type *tVar)
{
    UNION_S32 sTemp32;
    signed int sDeltaErr;

    tPID_Type tDVar;
    tDVar = *tVar;

    sDeltaErr = tDVar.sErr - tDVar.sErrLast;
    tDVar.sErrLast = tDVar.sErr;

    sTemp32.S32 = __builtin_mulus(tDVar.Kd, sDeltaErr);
    *tVar = tDVar;
    return (sTemp32.Words.High);
}
/*==========版权归张飞实战电子所有，违者必究！
```

微分项的话，就是计算相邻两次误差的差值，然后呢，乘以 Kd 系数。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
void PidControl(tPID_Type *tPidStruct)
{
    signed int sTemp;
    signed int sTempD;
    UNION_S32 sTemp32;

    tPID_Type tPidVar;
    tPidVar = *tPidStruct;
    sTemp = CalProportion(&tPidVar) + CalIntegration(&tPidVar);
    sTempD = CalDifferential(&tPidVar);
    sTemp32.S32 = (signed long)(sTemp) + (signed long)(sTempD);
    if (sTemp32.S32 > 32767)
        tPidVar.sOut = 32767;
    else if (sTemp32.S32 < -32767)
        tPidVar.sOut = -32767;
    else
        tPidVar.sOut = sTemp + sTempD;

    *tPidStruct = tPidVar;
}
```

然后，这个函数是一个汇总的函数，也就是分别计算出来比例，积分，微分三个项，再把他们加起来。

```
    tPID_Type tPidVar;
    tPidVar = *tPidStruct;
    sTemp = CalProportion(&tPidVar) + CalIntegration(&tPidVar);
    sTempD = CalDifferential(&tPidVar);
    sTemp32.S32 = (signed long)(sTemp) + (signed long)(sTempD);
    if (sTemp32.S32 > 32767)
        tPidVar.sOut = 32767;
    else if (sTemp32.S32 < -32767)
        tPidVar.sOut = -32767;
    else
        tPidVar.sOut = sTemp + sTempD;

    *tPidStruct = tPidVar;
}
```

然后呢，微分项是否有用，还要看三项加起来是不是大于 32767 了。如果说超了，就限制为 32767 或者-32767。

问：这里限制没有考虑到 D 吧。如果 PI 超了，D 已经没有用了？
答：是的，当然这仅是一种策略，PID 可以有很多种变形。

如果说没有超，那么就三个项一起作用。上面这些，就是 C 语言实现的 PID 控制。当然 PID 控制仍然有很多其它形式，大家感兴趣可以自己再去研究一下。

今天内容就到这里，下次课我们再来分析速度环对应的代码。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。